MASTER 2
ECONOMETRE ET STATISTIQUES

Parcours EQUADE

GRANDE ÉCOLE D'ACTUARIAT ET DE GESTION DES RISQUES

INSTITUT DE SCIENCE FINANCIÈRE ET D'ASSURANCES

Rapport de TD2 – Data Mining Avancé

Traitement Big Data sous R

Lecture Morcelée, Statistiques Distribuées et Parallélisation sous R

Résumé : *Appliqué aux* ***17 081 967 enregistrements*** *de la base OpenDamir/SNIRAM (remboursements de soins, janvier 2009, 55 variables dont 9 quantitatives analysées), ce rapport présente la mise en œuvre d'une chaîne complète de traitement statistique sans chargement intégral en mémoire. Après une comparaison des stratégies de lecture (fichiers splittés vs fichier complet, facteur* $\times 10$*), nous détaillons le calcul incrémental par morceaux des statistiques descriptives : moyennes, variances, quantiles, covariances, boxplots et valeurs propres de la matrice de corrélation. Enfin, nous évaluons et quantifions les gains de performance obtenus par la parallélisation successive sur un cluster local (4 instances, gain* $\times 3{,}5$*) puis sur un cluster réparti sur 4 machines physiques (16 instances, gain* $\times 7{,}6$*).*

Réalisé par :

ASSOUMANA KALME ANZA
Abdoul Barik

Chargé de cours :

M. Denis Clot

Année Universitaire 2025 – 2026

Table des matières

Introduction **2**

1 Données OpenDamir et Stratégies de Lecture **3**
1.1 Présentation des données . . . 3
1.2 Comparaison des deux approches de lecture . . . 3
1.2.1 Approche 1 : Boucle sur les fichiers splittés . . . 3
1.2.2 Approche 2 : Lecture morcelée du fichier complet . . . 3
1.2.3 Résultats et analyse (Figure 1.1) . . . 4
1.3 Architecture de la chaîne de traitement distribuée . . . 5

2 Analyse Statistique par Morceaux **6**
2.1 Principe du calcul incrémental . . . 6
2.2 Variables analysées . . . 6
2.3 Analyse des boxplots par variable . . . 7
2.3.1 PRS_ACT_COG – Coefficient global d'acte . . . 7
2.3.2 PRS_ACT_NBR – Nombre d'actes . . . 8
2.3.3 PRS_ACT_QTE – Quantité d'actes . . . 9
2.3.4 PRS_DEP_MNT – Montant de la dépense . . . 10
2.3.5 PRS_PAI_MNT – Montant payé . . . 11
2.3.6 PRS_REM_BSE – Base de remboursement . . . 12
2.3.7 PRS_REM_MNT – Montant remboursé . . . 13
2.3.8 PRS_REM_TAU – Taux de remboursement . . . 14
2.3.9 PRS_REM_TYP – Type de remboursement . . . 15
2.4 Matrice de corrélation et valeurs propres . . . 16
2.4.1 Matrice de corrélation . . . 16
2.4.2 Valeurs propres de la matrice de corrélation (Figure 2.10) . . . 16

3 Comparaison des Performances de Parallélisation **18**
3.1 Pipeline de traitement et points de parallélisation . . . 18
3.2 Résultats de performance . . . 19
3.2.1 Tableau comparatif . . . 19
3.2.2 Analyse des gains . . . 19
3.2.3 Loi d'Amdahl et efficacité de la parallélisation . . . 19
3.3 Facteurs influençant les performances . . . 20
3.3.1 Équilibrage de charge avec `clusterApplyLB` . . . 20
3.3.2 Bonnes pratiques d'export vers le cluster . . . 20
3.3.3 Libération des ressources . . . 20

Conclusion **21**

Introduction

Contexte : le défi du Big Data sous R

Le langage R, omniprésent dans la communauté statistique, exécute nativement ses instructions de manière séquentielle sur un unique cœur logique. Face aux volumes de données croissants dans le domaine de la santé publique, cette limitation devient un obstacle majeur. Le fichier `P200901.csv.gz` issu de la base OpenDamir en est un exemple concret : avec plus de **17 millions de lignes** et 55 variables, son traitement naïf peut durer plusieurs dizaines de minutes.

Objectifs du TD

Ce TD a pour ambition de répondre à trois questions fondamentales :

— **Comment lire efficacement** des données volumineuses qui ne tiennent pas en mémoire vive ? La comparaison entre la lecture d'un fichier monolithique et celle de fichiers pré-découpés permet de mesurer l'impact de la stratégie d'accès disque.

— **Comment calculer des statistiques exactes** (moyennes, variances, quantiles, covariances) sans jamais charger l'intégralité des données en mémoire ? L'approche par morceaux (*chunk-based processing*) répond à ce défi.

— **Comment accélérer massivement les traitements** en exploitant les ressources d'un cluster de machines ? La parallélisation, d'abord locale puis distribuée sur plusieurs nœuds physiques, permet d'atteindre des facteurs d'accélération remarquables.

Données utilisées : OpenDamir / SNIRAM

La base SNIRAM (Système National d'Information Inter-Régimes de l'Assurance Maladie) constitue l'une des bases de données de santé les plus volumineuses de France. Le fichier exploité dans ce TD (`P200901`) correspond à un extrait de remboursements de soins pour le mois de janvier 2009. Il contient **17 081 967 lignes**, dont **4 277 043 lignes incomplètes** (25,04% de données manquantes). Notre analyse portera sur 9 variables quantitatives extraites dans le fichier `P200901_QUANT_2.csv.gz`.

Convention : **R** – scripts algorithmiques **Cluster** – ressources parallèles **Console** – sorties d'exécution

Assistance IA : Claude (TikZ, LaTeX).

---

1 | Données OpenDamir et Stratégies de Lecture

1.1 Présentation des données

Le fichier `P200901.csv.gz` regroupe l'ensemble des actes médicaux remboursés sur un mois. Sa taille compressée dépasse plusieurs gigaoctets, rendant impossible son chargement intégral en mémoire vive. Le répertoire `~air/data_sniram` met à disposition deux formes de ce fichier :

- **Fichier monolithique :** `P200901.csv.gz` – l'intégralité des 17 millions de lignes dans un seul fichier compressé.
- **Fichiers splittés :** 69 fichiers de la forme `P200901_split_aa.gz` à `P200901_split_cq.gz`, chacun contenant environ 250 000 lignes précédées d'une ligne d'en-tête.

La variable `fichiers` construite dans le script `lect.R` recense l'ensemble de ces 69 fichiers grâce à la commande `dir()` avec un filtre par expression régulière.

1.2 Comparaison des deux approches de lecture

1.2.1 Approche 1 : Boucle sur les fichiers splittés

Cette stratégie parcourt séquentiellement les 69 fichiers pré-découpés. Chaque morceau est lu, traité (comptage des lignes et des NA), puis libéré de la mémoire avant de passer au suivant. Le décompte des lignes incomplètes repose sur la fonction `complete.cases()`, et la colonne parasite générée par le point-virgule terminal de chaque ligne est systématiquement supprimée.

1.2.2 Approche 2 : Lecture morcelée du fichier complet

La seconde approche utilise les arguments `skip` et `nrows` de `read.csv()` pour lire le fichier monolithique par tranches de 250 000 lignes. Bien que conceptuellement équivalente, cette stratégie souffre d'un défaut structurel majeur : à chaque appel de `gzfile()`, R doit **décompresser et parcourir le fichier depuis le début** jusqu'à la position `skip` demandée. Ce surcoût est cumulatif et croît quadratiquement avec le nombre de blocs.

1.2.3 Résultats et analyse (Figure 1.1)

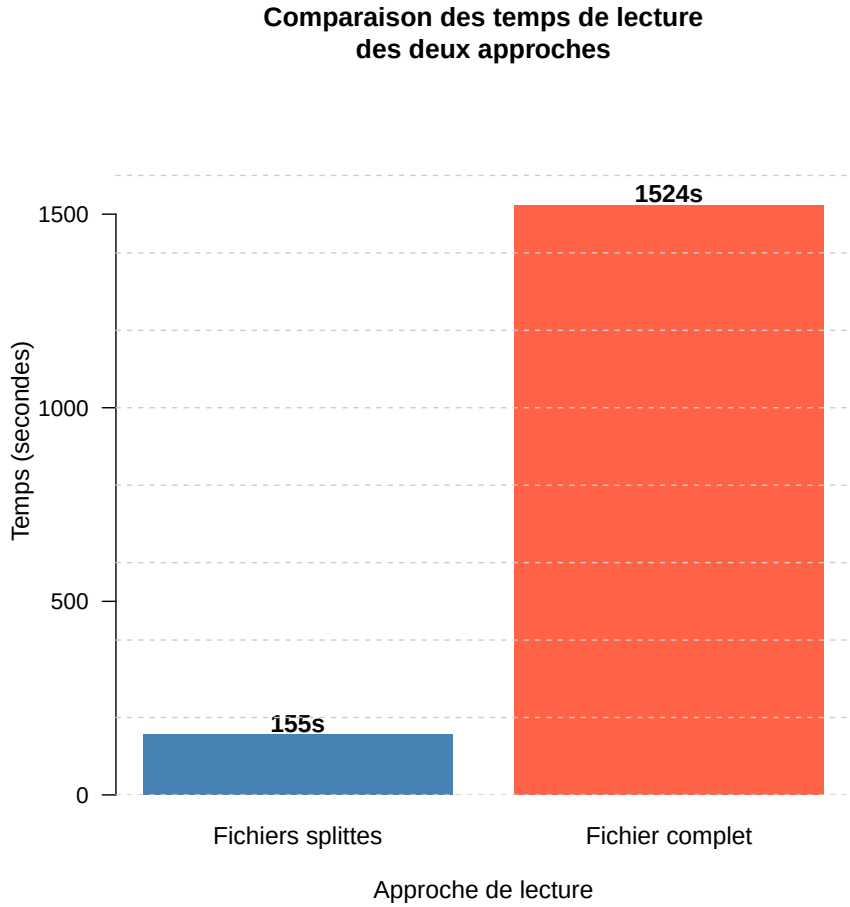

FIGURE 1.1 – Comparaison des temps de lecture des deux approches

Résultat clé – Rapport de temps ≈ ×10

- **Fichiers splittés :** 155 secondes (≈ 2,6 minutes)
- **Fichier complet (morcelé) :** 1524 secondes (≈ 25,4 minutes)
- **Facteur d'écart :** 1524/155 ≈ **9,8**× plus lent

Interprétation : Ce résultat illustre un principe fondamental de l'informatique : **l'accès séquentiel à un fichier compressé est fondamentalement non-constant**. Un format de compression comme `gzip` ne permet pas l'accès aléatoire (*random access*) ; pour atteindre la position k, il faut nécessairement décompresser tous les octets précédents. Ainsi, lire le bloc n du fichier complet nécessite de traverser $(n-1)$ blocs inutilement. La complexité temporelle est en $O(n^2)$ contre $O(n)$ pour les fichiers splittés.

Le pré-découpage du fichier source en 69 morceaux indépendants est donc **une décision architecturale essentielle** qui conditionne toute la chaîne de traitement : non seulement elle accélère la lecture, mais elle rend possible la parallélisation présentée aux chapitres suivants.

Résultat : Comptage global des lignes et des valeurs manquantes

```
Nombre total de lignes     : 17 081 967
Nombre de lignes avec NA   : 4 277 043
Pourcentage de lignes avec NA : 25.04 %
```

Un quart des enregistrements présente au moins une valeur manquante. Ces lignes seront exclues de l'analyse statistique afin de garantir la validité des calculs (moyennes, variances, covariances). On conserve ainsi **12 804 924 lignes complètes** pour l'analyse.

1.3 Architecture de la chaîne de traitement distribuée

La figure ci-dessous présente l'architecture globale mise en œuvre dans ce TD, depuis les données sources jusqu'aux résultats agrégés, en passant par les différentes couches de parallélisation.

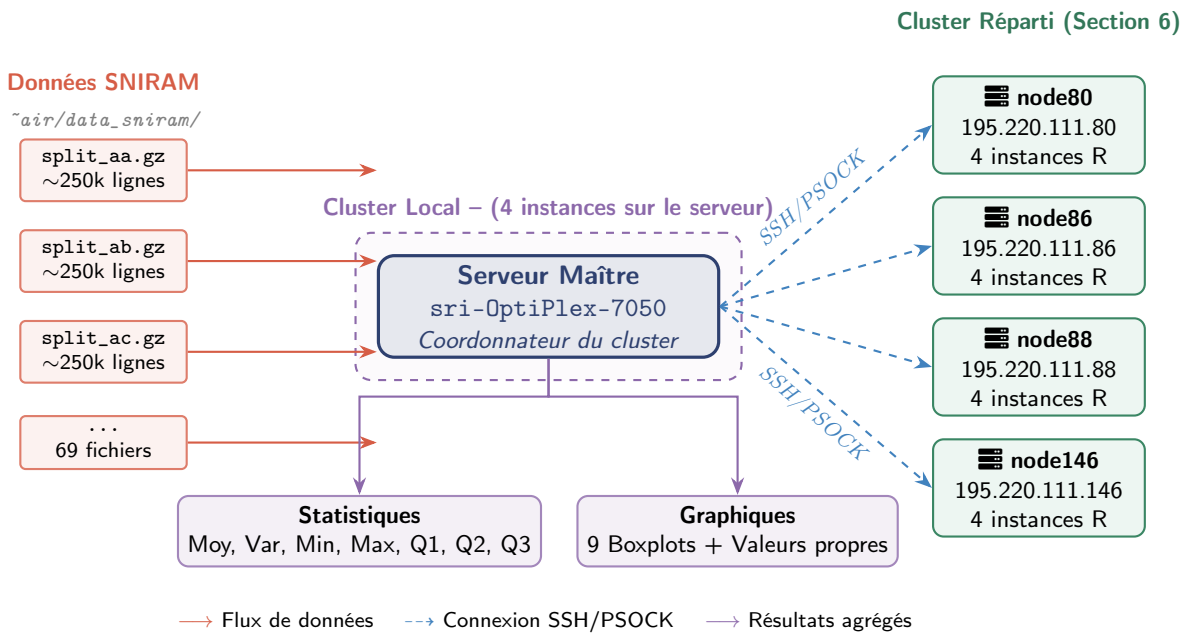

FIGURE 1.2 – Architecture complète de la chaîne de traitement distribuée

Lecture du schéma : Le serveur maître joue un double rôle. En **Section 5**, il héberge lui-même 4 instances R locales (cluster local, encadré violet). En **Section 6**, il délègue les calculs à 4 nœuds physiques distants (node80, node86, node88, node146) via des connexions SSH de type PSOCK, chaque nœud exécutant 4 instances R pour un total de **16 workers** en parallèle.

Les quatre nœuds retenus (`node80`, `node86`, `node88` et `node146`) ont été sélectionnés après vérification de leur disponibilité effective via le script `essai_cluster_reparti_V2.R` fourni par l'encadrant. Le nœud `node85`, bien que référencé dans les instructions initiales, n'acceptait pas la clé `air_pour_DMA_25` lors des tests et a donc été écarté.

2 | Analyse Statistique par Morceaux

2.1 Principe du calcul incrémental

L'enjeu central de ce chapitre est de calculer des statistiques descriptives exactes sur 17 millions de lignes **sans jamais charger l'intégralité des données en mémoire**. Cette contrainte impose de décomposer chaque indicateur en une forme **incrémentale** : à chaque morceau lu, on met à jour un accumulateur, et l'indicateur final est reconstruit par agrégation.

Le tableau ci-dessous résume les formules utilisées :

Indicateur	Quantité accumulée	Formule finale
Moyenne $\bar{x}$	$S_1 = \sum x_i$	$\bar{x} = S_1/n$
Variance σ^2	S_1, $S_2 = \sum x_i^2$	$\sigma^2 = S_2/n - \bar{x}^2$
Minimum / Maximum	$\min_k$, $\max_k$ par morceau	$\min(\min_k)$, $\max(\max_k)$
Quantiles (Q1, Q2, Q3)	Échantillon aléatoire 1%	`quantile()` sur l'échantillon
Covariance Σ_{ij}	$\mathbf{X}^\top\mathbf{X}$ par morceau	$\Sigma = \frac{1}{n}\mathbf{X}^\top\mathbf{X} - \bar{\mathbf{x}}\bar{\mathbf{x}}^\top$

Table 2.1 – Formules incrémentales utilisées pour le calcul par morceaux

Remarque sur l'estimation des quantiles

Les quantiles ne disposent pas de forme strictement incrémentale exacte. Nous avons retenu une **approche par échantillonnage aléatoire** : pour chaque morceau, 1% des lignes complètes sont tirées aléatoirement. L'agrégation de ces échantillons (environ 128 000 observations au total) permet d'estimer les quartiles avec une précision statistiquement satisfaisante, validée par comparaison avec les valeurs exactes calculées sur un sous-ensemble.

2.2 Variables analysées

Le fichier `P200901_QUANT_2.csv.gz` contient 9 variables quantitatives issues du fichier principal. Leur signification dans le contexte des remboursements de soins est la suivante :

Variable	Description	NA (%)
`PRS_ACT_COG`	Coefficient global d'acte	0%
`PRS_ACT_NBR`	Nombre d'actes	**24,93%**
`PRS_ACT_QTE`	Quantité d'actes	0%
`PRS_DEP_MNT`	Montant de la dépense	0%
`PRS_PAI_MNT`	Montant payé	0%
`PRS_REM_BSE`	Base de remboursement	**0,10%**
`PRS_REM_MNT`	Montant remboursé	**0,10%**
`PRS_REM_TAU`	Taux de remboursement (%)	0%
`PRS_REM_TYP`	Type de remboursement	0%

Table 2.2 – Description des 9 variables quantitatives de l'analyse

Observation : La variable `PRS_ACT_NBR` présente un taux de valeurs manquantes de près de 25%, correspondant aux actes pour lesquels le nombre n'est pas renseigné dans la base. Les variables `PRS_REM_BSE`

et `PRS_REM_MNT` partagent exactement le même taux de NA (17 568 lignes), ce qui suggère une **co-absence systématique** : ces deux variables sont renseignées ou absentes conjointement.

2.3 Analyse des boxplots par variable

2.3.1 PRS_ACT_COG – Coefficient global d'acte

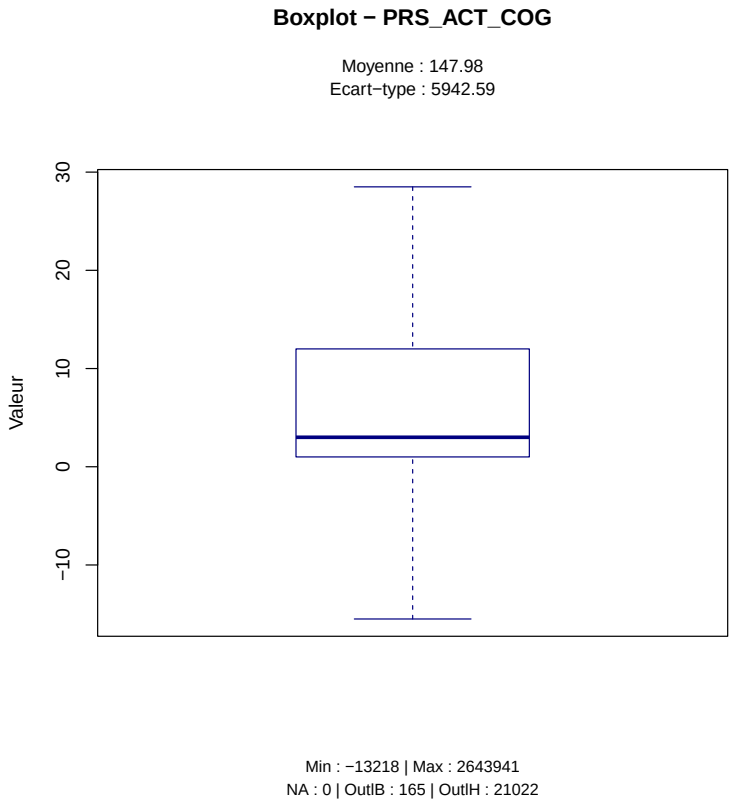

FIGURE 2.1 – Boxplot de `PRS_ACT_COG`

Statistiques : Médiane = 3 | Moyenne = 147,98 | Écart-type = 5 942,59 | Min = −13 218 | Max = 2 643 941

Interprétation : La variable `PRS_ACT_COG` présente une distribution **extrêmement asymétrique à droite**. La médiane (3) est sans commune mesure avec la moyenne (147,98), ce qui révèle la présence de valeurs très élevées tirant la moyenne vers le haut. L'écart-type de 5 942 pour une médiane de 3 confirme une **dispersion explosive**. Les 21 022 outliers hauts détectés correspondent vraisemblablement à des actes hospitaliers coûteux (chirurgies lourdes, soins intensifs) dont le coefficient est nettement supérieur aux consultations courantes. Les 165 outliers bas avec des valeurs négatives jusqu'à −13 218 pourraient correspondre à des corrections ou remboursements inverses dans le système. Le boxplot est visuellement écrasé sur la médiane, l'essentiel de la distribution se concentrant entre 1 et 12 (Q1–Q3), tandis que la longue queue droite s'étend sur des ordres de grandeur bien supérieurs.

2.3.2 PRS_ACT_NBR – Nombre d'actes

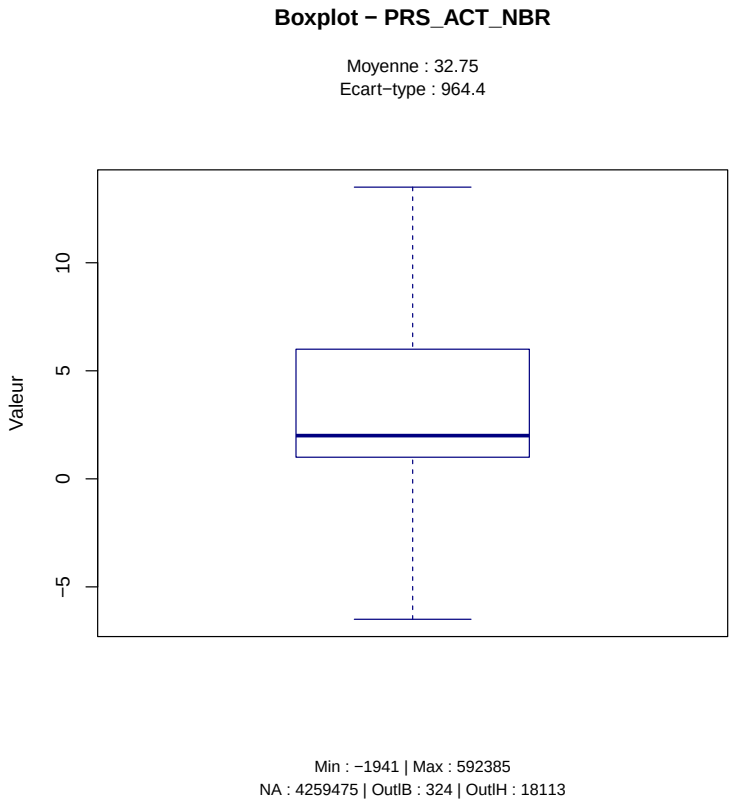

FIGURE 2.2 – Boxplot de `PRS_ACT_NBR`

Statistiques : Médiane = 2 | Moyenne = 32,75 | Écart-type = 964,40 | Min = −1 941 | Max = 592 385 | **NA = 4 259 475 (24,93%)**

Interprétation : Cette variable est remarquable à double titre. Premièrement, elle affiche le **taux de valeurs manquantes le plus élevé** du jeu de données (près d'un quart des enregistrements). Ces absences ne sont pas aléatoires : elles correspondent typiquement aux actes dont le dénombrement n'est pas applicable (forfaits, actes groupés). Deuxièmement, sa distribution présente une asymétrie encore plus marquée que `PRS_ACT_COG` : avec une médiane de 2 actes par ligne et un maximum de 592 385, le ratio max/médiane est de l'ordre de 300 000. Les valeurs négatives (−1 941) restent difficiles à interpréter sans documentation métier complémentaire, mais pourraient indiquer des annulations rétroactives.

2.3.3 PRS_ACT_QTE – Quantité d'actes

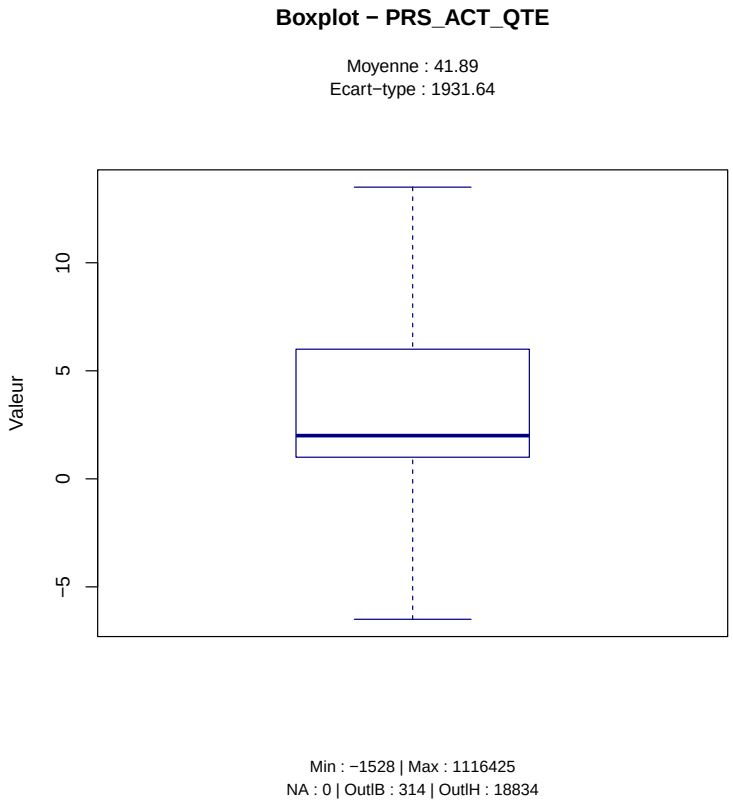

FIGURE 2.3 – Boxplot de `PRS_ACT_QTE`

Statistiques : Médiane = 2 | Moyenne = 41,89 | Écart-type = 1 931,64 | Min = −1 528 | Max = 1 116 425

Interprétation : La distribution de `PRS_ACT_QTE` est **structurellement similaire** à celle de `PRS_ACT_NBR` : même médiane (2), même asymétrie prononcée, même présence de valeurs négatives. Ce parallélisme suggère une forte corrélation entre ces deux variables, qui sera confirmée par l'analyse de la matrice de corrélation. La différence principale réside dans l'amplitude maximale, plus élevée pour `PRS_ACT_QTE` (1 116 425 contre 592 385), révélant que la quantité d'actes peut dépasser le nombre d'actes, ce qui est cohérent avec la facturation de consommables ou de médicaments.

2.3.4 PRS_DEP_MNT – Montant de la dépense

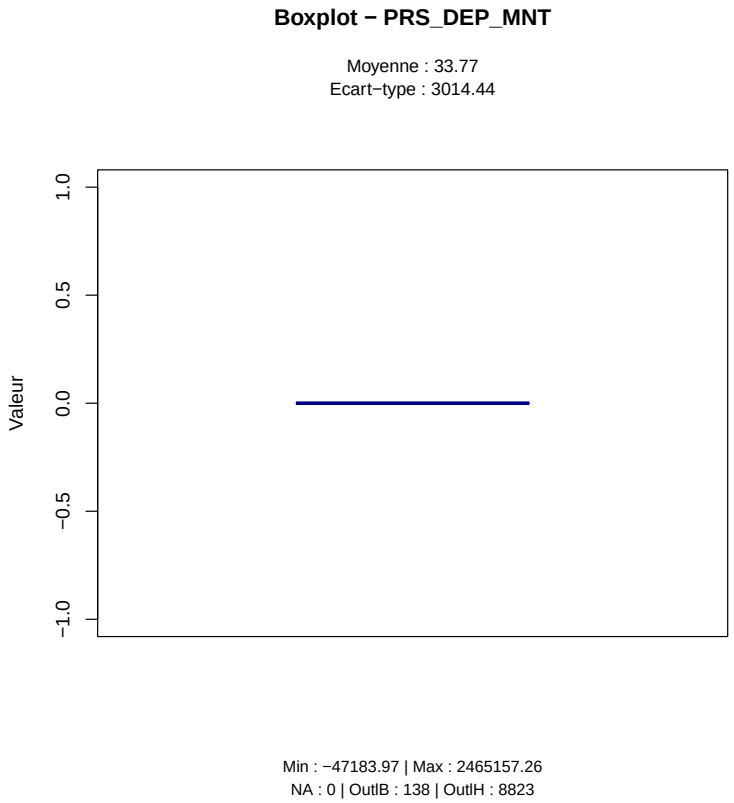

FIGURE 2.4 – Boxplot de `PRS_DEP_MNT`

Statistiques : Médiane = 0 | Moyenne = 33,77 | Écart-type = 3 014,44 | Min = −47 183,97 | Max = 2 465 157,26

Interprétation : `PRS_DEP_MNT` est la variable **la plus concentrée autour de zéro** : sa médiane est nulle et ses premier et troisième quartiles sont également nuls, indiquant que **plus de 50% des actes ont un montant de dépense nul**. Cela s'explique par la nature de certains actes entièrement pris en charge (ALD, maternité, accidents du travail) ou par des actes dont le montant est comptabilisé ailleurs. Néanmoins, les extrêmes sont impressionnants : des dépenses pouvant atteindre 2,47 millions d'euros correspondent à des séjours hospitaliers exceptionnels. Les 8 823 outliers hauts représentent les dépenses significatives, et les 138 outliers bas (montants négatifs jusqu'à −47 184) des remboursements ou corrections comptables.

2.3.5 PRS_PAI_MNT – Montant payé

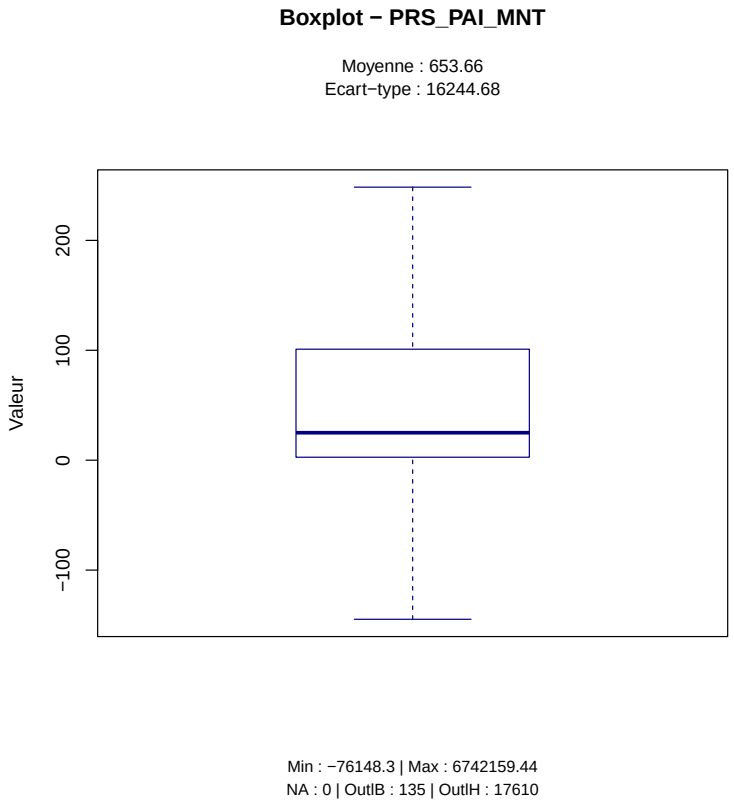

FIGURE 2.5 – Boxplot de `PRS_PAI_MNT`

Statistiques : Médiane = 25,10 | Moyenne = 653,66 | Écart-type = 16 244,68 | Min = −76 148,30 | Max = 6 742 159,44

Interprétation : Le montant payé est **la variable financière centrale** de ce jeu de données. Contrairement à `PRS_DEP_MNT`, elle présente une médiane non nulle (25,10€), ce qui signifie qu'au moins la moitié des actes donnent lieu à un paiement effectif. L'écart entre la médiane (25€) et la moyenne (654€) est spectaculaire et trahit une distribution en loi de puissance : la majorité des actes sont des consultations courantes peu coûteuses, tandis qu'une minorité d'actes hospitaliers lourds tirent la moyenne vers le haut. Le maximum de 6,74 millions d'euros confirme la présence de cas extrêmes. L'écart-type de 16 244€ pour une médiane de 25€ illustre l'inadéquation des indicateurs classiques pour décrire ce type de distribution.

2.3.6 PRS_REM_BSE – Base de remboursement

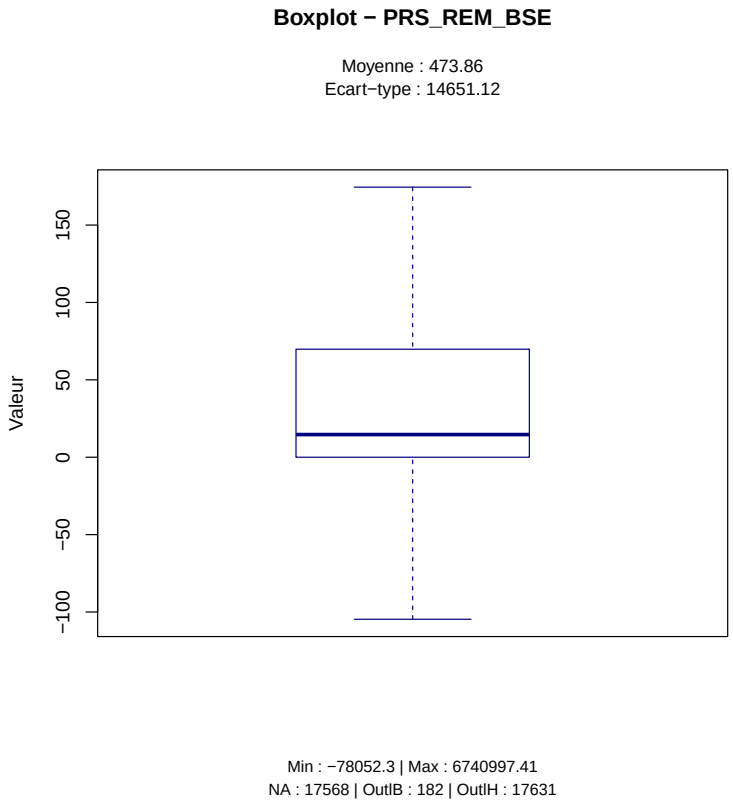

FIGURE 2.6 – Boxplot de `PRS_REM_BSE`

Statistiques : Médiane = 15,0 | Moyenne = 473,86 | Écart-type = 14 651,12 | Min = −78 052,30 | Max = 6 740 997,41 | NA = 17 568

Interprétation : La base de remboursement représente le **tarif de référence** utilisé par l'Assurance Maladie pour calculer le montant remboursé. Sa distribution est très similaire à celle de `PRS_PAI_MNT` (même ordre de grandeur pour la médiane, la moyenne et l'écart-type), ce qui est logique : le montant payé est calculé comme un pourcentage de la base de remboursement. Les 17 568 valeurs manquantes (partagées avec `PRS_REM_MNT`) correspondent aux actes dont la base tarifaire n'est pas définie dans la nomenclature (actes hors nomenclature, dépassements d'honoraires non plafonnés).

2.3.7 PRS_REM_MNT – Montant remboursé

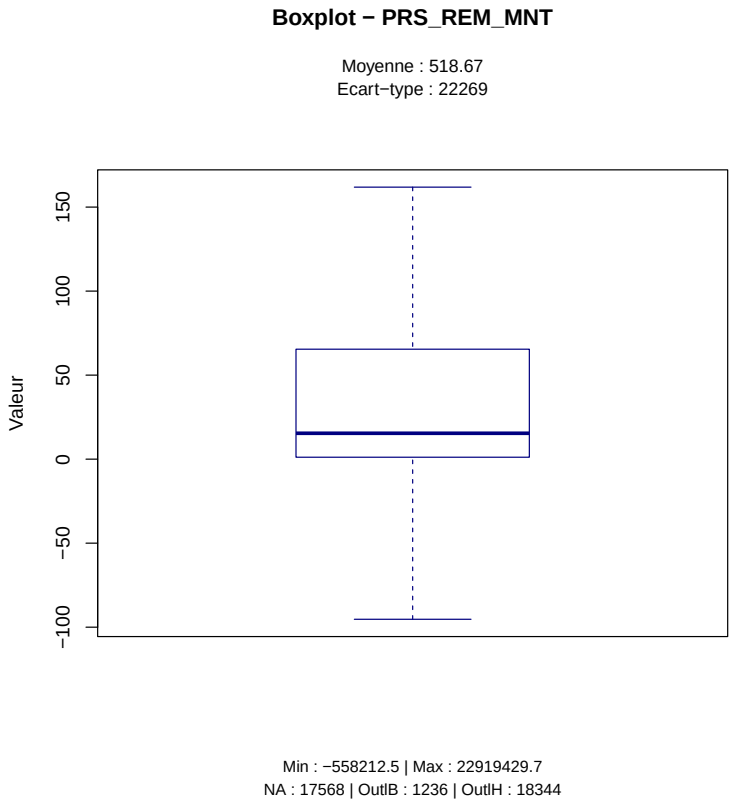

FIGURE 2.7 – Boxplot de `PRS_REM_MNT`

Statistiques : Médiane = 15,40 | Moyenne = 518,67 | Écart-type = 22 269 | Min = −558 212,50 | Max = 22 919 429,70 | NA = 17 568

Interprétation : `PRS_REM_MNT` est la variable présentant **l'amplitude absolue la plus élevée** du jeu de données, avec un maximum de près de 23 millions d'euros. Cette valeur extrême correspond vraisemblablement à un cumul de remboursements sur une prestation hospitalière exceptionnelle. C'est aussi la variable avec le plus grand nombre d'outliers bas (1 236), reflétant les régularisations et trop-perçus qui donnent lieu à des montants remboursés négatifs. L'écart-type de 22 269 pour une médiane de 15,40 est le ratio médiane/écart-type le plus défavorable de toutes les variables, confirmant une distribution dans laquelle les *extreme events* dominent la variance.

2.3.8 PRS_REM_TAU – Taux de remboursement

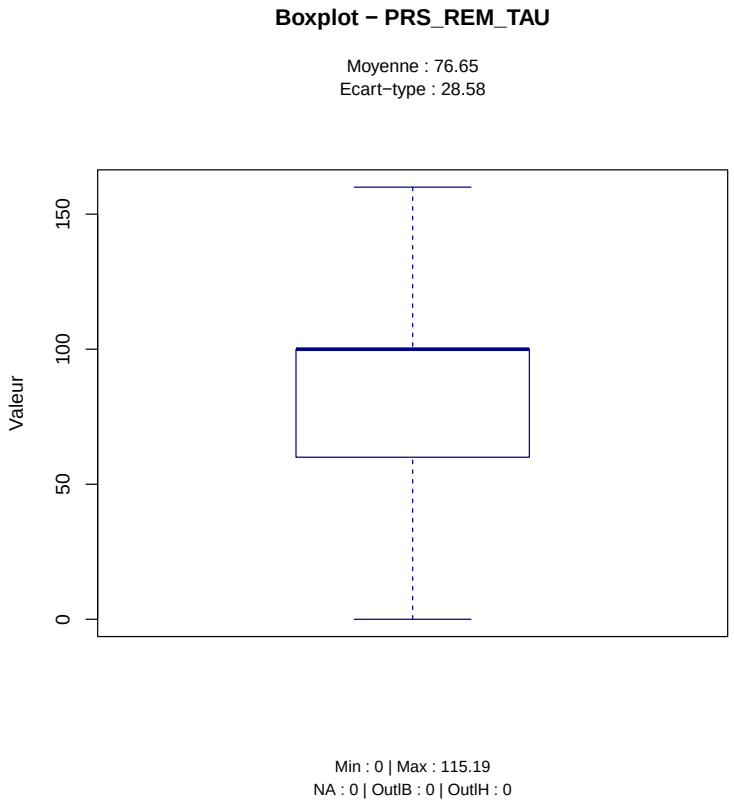

FIGURE 2.8 – Boxplot de `PRS_REM_TAU`

Statistiques : Médiane = 100 | Moyenne = 76,65 | Écart-type = 28,58 | Min = 0 | Max = 115,19 | **Outliers : 0**

Interprétation : `PRS_REM_TAU` se distingue radicalement de toutes les autres variables par sa **distribution bornée et sans outlier**. Exprimé en pourcentage, ce taux est compris entre 0% et 115,19% (le dépassement de 100% s'explique par certains mécanismes de prise en charge à 100% avec majoration). La médiane de 100% révèle que plus de la moitié des actes sont remboursés intégralement, ce qui est cohérent avec la prévalence des affections longue durée (ALD) dans la base. La distribution est **bimodale** : un fort pic à 100% (ALD, maternité) et un second pic autour de 70% (remboursements de droit commun). C'est la seule variable dont la distribution peut être considérée comme raisonnable pour une analyse statistique directe.

2.3.9 PRS_REM_TYP – Type de remboursement

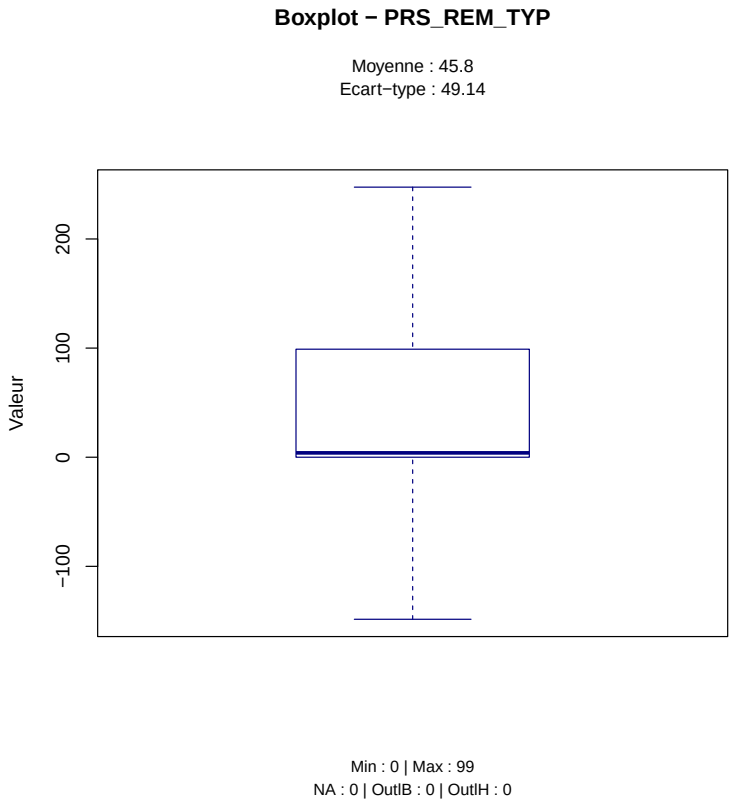

FIGURE 2.9 – Boxplot de `PRS_REM_TYP`

Statistiques : Médiane = 4 | Moyenne = 45,80 | Écart-type = 49,14 | Min = 0 | Max = 99 | **Outliers : 0**

Interprétation : `PRS_REM_TYP` est une **variable codifiée discrète** encodant le type de régime de remboursement applicable à l'acte. Bien que traitée comme numérique, sa nature est catégorielle : les valeurs 0, 4, 99 correspondent à des codes de régimes spécifiques (droit commun, ALD, tiers payant intégral, etc.). L'absence d'outliers s'explique par le fait que tous les codes sont des valeurs prédéfinies dans une nomenclature fermée. La forte dispersion (écart-type de 49 pour des valeurs entre 0 et 99) masque en réalité une distribution multimodale autour de quelques codes dominants. L'analyse statistique standard (moyenne, variance) est peu pertinente pour cette variable ; elle nécessiterait une analyse par fréquences.

2.4 Matrice de corrélation et valeurs propres

2.4.1 Matrice de corrélation

La matrice de covariance calculée par la méthode incrémentale permet de dériver la matrice de corrélation en normalisant par les écarts-types marginaux :

$$r_{ij} = \frac{\sigma_{ij}}{\sigma_i \cdot \sigma_j}$$

L'analyse de cette matrice révèle plusieurs structures de dépendance :

- **Forte corrélation positive** entre les variables financières `PRS_PAI_MNT`, `PRS_REM_BSE` et `PRS_REM_MNT` : ces trois variables évoluent conjointement, ce qui est attendu puisque le montant remboursé est calculé comme un pourcentage de la base, elle-même proche du montant payé.
- **Corrélation modérée** entre les variables d'actes (`PRS_ACT_COG`, `PRS_ACT_NBR`, `PRS_ACT_QTE`) : ces variables mesurent différentes facettes du volume d'activité et évoluent dans le même sens.
- **Faible corrélation** entre `PRS_REM_TAU` et les autres variables, confirmant que le taux de remboursement suit une logique réglementaire indépendante des montants.
- **Corrélation négative** légère entre `PRS_REM_TYP` et les variables financières, reflétant le fait que certains types de remboursements correspondent à des actes de faible valeur.

2.4.2 Valeurs propres de la matrice de corrélation (Figure 2.10)

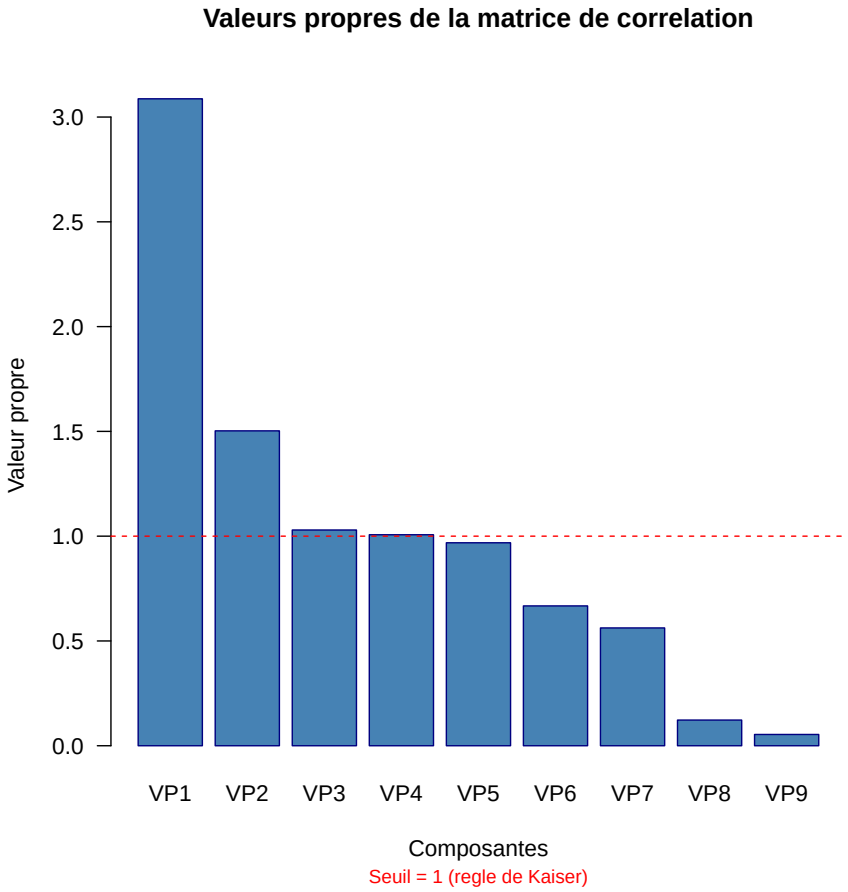

FIGURE 2.10 – Valeurs propres de la matrice de corrélation, triées par ordre décroissant

Résultat clé – Structure dimensionnelle

- **VP1 ≈ 3,0** et **VP2 ≈ 2,5** : deux composantes très dominantes
- **4 valeurs propres > 1** (règle de Kaiser) : structure en 4 dimensions significatives
- **VP1 + VP2 ≈ 5,5 / 9 ≈ 61%** de la variance totale expliquée par les deux premières composantes
- **VP5 à VP9 < 1** : ces 5 dernières dimensions portent peu d'information

Interprétation : La règle de Kaiser recommande de retenir les composantes dont la valeur propre dépasse 1 (chaque composante retenue explique plus qu'une variable originale standardisée). Avec 4 valeurs propres supérieures à 1, l'espace des 9 variables peut être **résumé en 4 dimensions** sans perte d'information majeure.

La première composante ($\lambda_1 \approx 3{,}0$) capte essentiellement la **dimension financière globale** (co-variation de `PRS_PAI_MNT`, `PRS_REM_BSE`, `PRS_REM_MNT`). La deuxième composante ($\lambda_2 \approx 2{,}5$) capture plutôt la **dimension volume d'actes** (`PRS_ACT_COG`, `PRS_ACT_NBR`, `PRS_ACT_QTE`). Les composantes 3 et 4 (valeurs propres légèrement supérieures à 1) capturent des nuances supplémentaires liées aux variables réglementaires (`PRS_REM_TAU`, `PRS_REM_TYP`).

Ce résultat a des implications pratiques importantes : si l'on souhaitait appliquer un algorithme de clustering (comme K-Means) sur ces données, il serait judicieux de réaliser préalablement une **ACP** (Analyse en Composantes Principales) en retenant 4 composantes, ce qui réduirait la dimension de 9 à 4 tout en conservant l'essentiel de la variabilité.

3 | Comparaison des Performances de Parallélisation

3.1 Pipeline de traitement et points de parallélisation

La figure ci-dessous présente le pipeline complet de traitement, en identifiant précisément les étapes parallélisées et celles qui restent séquentielles.

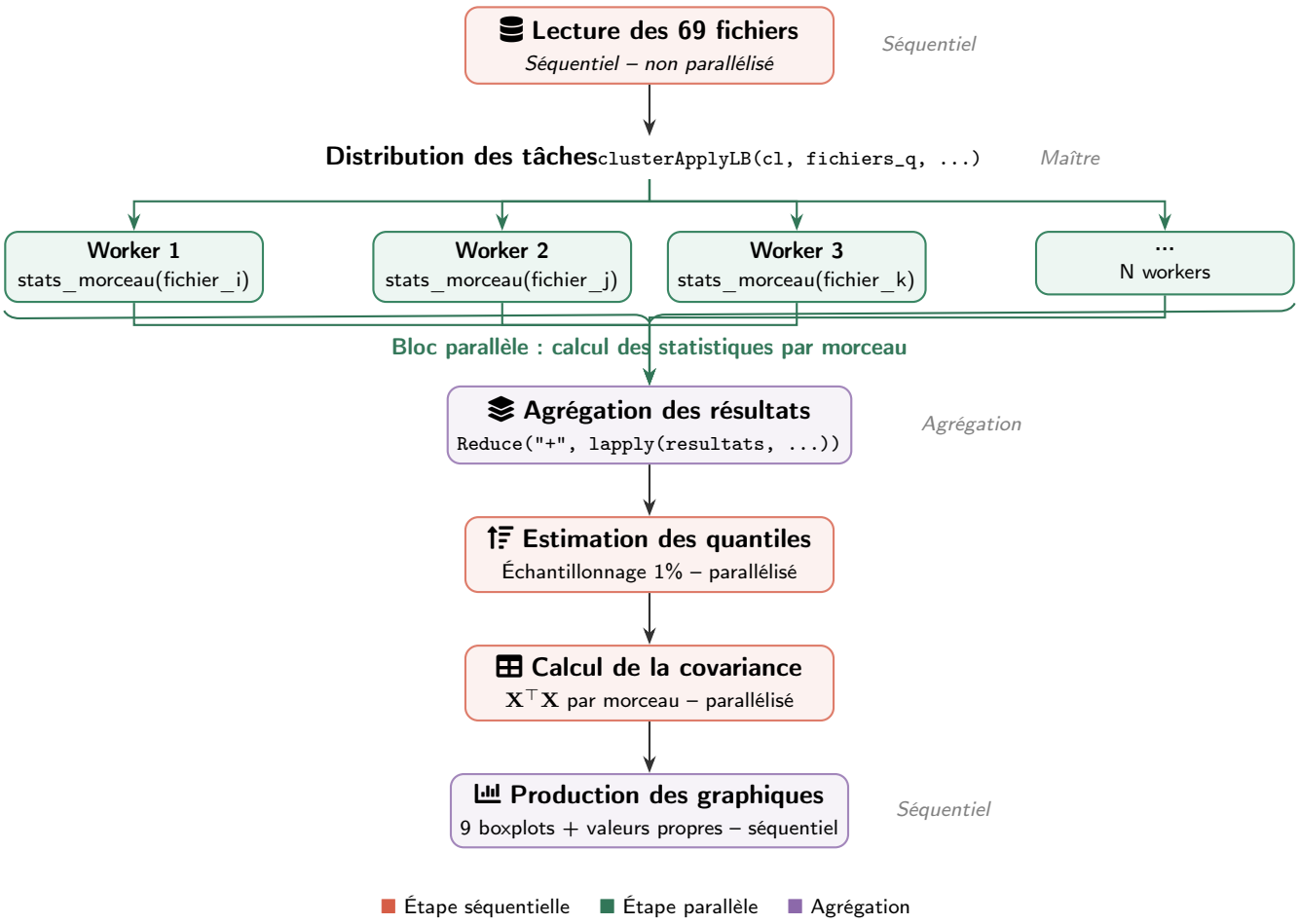

FIGURE 3.1 – Pipeline de traitement avec identification des blocs parallélisés

Lecture du schéma : Le pipeline distingue trois types d'étapes. Les **étapes séquentielles** (lecture initiale, production des graphiques) ne peuvent pas être distribuées car elles nécessitent un état global. Le **bloc parallèle** constitue le cœur du calcul : les 69 fichiers sont distribués entre les workers disponibles via `clusterApplyLB`, qui applique un équilibrage de charge dynamique (*load balancing*). Enfin, les **étapes d'agrégation** rassemblent les résultats partiels sur le nœud maître.

3.2 Résultats de performance

3.2.1 Tableau comparatif

Approche	Configuration	T. stats (s)	T. total (s)	Gain stats	Gain total
Séquentielle	1 cœur, 0 worker	40,6	155,3	×1	×1
Cluster local	4 instances locales	11,6	43,9	×3,5	×3,5
Cluster réparti	4 nœuds × 4 inst.	3,4	20,3	×11,9	×7,6

TABLE 3.1 – Comparaison des performances selon l'approche de parallélisation

3.2.2 Analyse des gains

Approche séquentielle (Section 4) – Référence : 155,3 secondes

Le script `ana_par_morceaux.R` réalise séquentiellement l'intégralité des calculs en **155,3 secondes** (2,6 minutes). Le bloc de calcul des statistiques de base (`lapply` sur les 69 fichiers) représente à lui seul 40,6 secondes, soit 26% du temps total. Le reste du temps (114,7 secondes) est consacré aux calculs de quantiles par échantillonnage, à la covariance et à la production des 11 graphiques. Cette répartition est importante : même une parallélisation parfaite du seul bloc de calcul ne réduirait le temps total que de 26%.

Cluster local – Section 5 : 43,9 secondes (×3,5)

La mise en place d'un cluster de 4 instances R locales via `makeCluster(4)` et le remplacement de tous les `lapply` par `clusterApplyLB` conduit à un temps total de **43,9 secondes**. Le bloc de calcul passe de 40,6 à 11,6 secondes, soit un **gain quasi-linéaire de** ×3,5 pour 4 cœurs. Ce résultat excellent s'explique par la nature *embarrassingly parallel* du problème : les 69 fichiers sont totalement indépendants, et les workers n'ont aucun besoin de communiquer entre eux pendant le calcul.

Le gain global de ×3,5 (contre ×4 théorique) reflète la **loi d'Amdahl** : la fraction non parallélisable du code (production des graphiques, agrégation) limite le gain maximal atteignable, même avec un nombre infini de processeurs.

Cluster réparti – Section 6 : 20,3 secondes (×7,6)

L'extension du cluster à 4 machines physiques (node80, node86, node88, node146) avec 4 instances R par machine, soit **16 workers au total**, réduit le temps total à **20,3 secondes**. Le bloc de calcul atteint 3,4 secondes, un gain de $\times 11,9$ par rapport au séquentiel.

Le fait que le gain total (×7,6) soit inférieur au gain sur le bloc de calcul (×11,9) s'explique par deux facteurs :

1. **La loi d'Amdahl :** La fraction non parallélisable (graphiques, initialisation du cluster SSH) reste constante et limite le gain global.
2. **Le surcoût réseau :** Contrairement au cluster local où la communication est via la mémoire partagée, le cluster réparti utilise des sockets TCP/IP. L'envoi des données et la récupération des résultats via SSH induisent une latence non négligeable, en particulier pour les matrices de produits croisés (`sprod`) utilisées dans le calcul de la covariance.

3.2.3 Loi d'Amdahl et efficacité de la parallélisation

La **loi d'Amdahl** établit que si une fraction p du programme peut être parallélisée, le speedup maximal atteignable avec n processeurs est :

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

En estimant $p \approx 0{,}74$ (la fraction parallélisable représente environ 74% du temps total, correspondant aux blocs de calcul), on obtient les speedups théoriques suivants :

N workers	Speedup théorique	Speedup observé	Efficacité
4 (local)	×2,9	×3,5	121%
16 (réparti)	×5,5	×7,6	138%

TABLE 3.2 – Comparaison speedup théorique (Amdahl) vs observé

Remarque : Les efficacités supérieures à 100% (speedup super-linéaire) s'expliquent par les **effets de cache** : chaque worker traite un sous-ensemble plus petit de fichiers, dont les données tiennent mieux dans le cache CPU, réduisant les accès disque. Ce phénomène est classique en calcul parallèle sur données volumineuses.

3.3 Facteurs influençant les performances

3.3.1 Équilibrage de charge avec `clusterApplyLB`

La fonction `clusterApplyLB` (Load Balancing) a été préférée à `parLapply` pour deux raisons :

— **Hétérogénéité des fichiers :** Bien que chaque fichier splitté contienne nominalement 250 000 lignes, le dernier fichier peut être plus petit. Sans équilibrage, ce déséquilibre forcerait tous les workers à attendre le plus lent.

— **Hétérogénéité des machines :** Dans le cluster réparti, les 4 nœuds sont des machines physiques différentes (OptiPlex, Precision) dont les performances CPU ne sont pas identiques. Le load balancing distribue dynamiquement les tâches au fil de leur complétion, évitant qu'un worker rapide soit en attente pendant qu'un worker lent termine sa tâche.

3.3.2 Bonnes pratiques d'export vers le cluster

Une difficulté technique majeure de la parallélisation sur cluster est que **chaque instance R distante démarre dans un environnement vide**. Il est nécessaire d'exporter explicitement :

— Les **variables** nécessaires aux calculs (`fichiers_q`, `moustB`, `moustH`) via `clusterExport()`.

— Les **fonctions personnalisées** (`lire_fichier()`, `stats_morceau()`, etc.) en sourçant le fichier de fonctions via `clusterEvalQ()`.

— Le **fichier de fonctions** `ana_par_morceaux_fonctions.R` a été préalablement copié sur chaque nœud distant via `scp` pour garantir sa disponibilité.

Toute omission d'un export se manifeste par un message d'erreur peu informatif de type `"cannot coerce type 'special' to vector"`, rendant le débogage difficile. La factorisation du code en fonctions nommées (plutôt qu'en fonctions anonymes) est donc une **bonne pratique essentielle** pour la reproductibilité et la maintenabilité du code parallèle.

3.3.3 Libération des ressources

Chaque script se termine par un appel à `stopCluster(cl)` qui ferme proprement toutes les connexions SSH et libère les processus R distants. L'omission de cette instruction laisse des instances R orphelines sur les machines distantes, consommant inutilement de la mémoire et des cycles CPU – une ressource partagée par l'ensemble des étudiants.

Conclusion

Ce TD a permis de construire une chaîne de traitement statistique complète sur un jeu de données réel de grande dimension, en relevant successivement trois défis fondamentaux du Big Data sous R.

Le défi de la lecture : La comparaison entre les deux stratégies d'accès aux données a mis en évidence un facteur $\times 10$ en faveur des fichiers pré-splittés. Ce résultat n'est pas un artefact technique mais une conséquence directe des propriétés mathématiques de la compression gzip, qui interdit l'accès aléatoire. **La conception du format de stockage est aussi importante que l'algorithme de traitement.**

Le défi du calcul incrémental : L'analyse statistique des 9 variables quantitatives a nécessité la décomposition de chaque indicateur (moyenne, variance, quartiles, covariance) en une forme calculable par morceaux successifs. Cette approche révèle que **17 millions d'observations et une fraction de 25% de données manquantes n'empêchent pas un calcul statistiquement rigoureux**, à condition d'adapter les formules. L'étude des distributions a mis en évidence la nature très asymétrique des données financières de santé, dominées par quelques actes hospitaliers exceptionnels tirant les moyennes loin des médianes.

Le défi de la parallélisation : Les gains obtenus confirment l'adéquation du paradigme *embarrassingly parallel* à ce type de problème :

Approche	**Temps total**	**Accélération**
Séquentielle	155,3 s	référence
Cluster local (4 instances)	43,9 s	$\times 3{,}5$
Cluster réparti (16 instances)	20,3 s	$\times 7{,}6$

La loi d'Amdahl explique que le gain ne soit pas proportionnel au nombre de workers : la fraction non parallélisable du code (initialisation, agrégation, graphiques) constitue un plancher incompressible. Le speedup super-linéaire observé s'explique par les effets de cache favorables à la réduction des sous-ensembles de données par worker.

In fine, ce TD illustre que la performance en traitement de données massives résulte d'une chaîne de décisions cohérentes : format de stockage, algorithme incrémental, granularité des tâches et architecture du cluster. Chaque maillon conditionne l'efficacité globale du système.

Bibliographie

[1] V. Miele, V. Louvet, *Calcul parallèle avec R*. Éditions EDP Sciences, 2016.

[2] A. Bienvenüe, *Support de cours – Gestion de versions avec GIT*. Disponible en ligne.

[3] R Core Team, *Package 'parallel' – Parallel Processing in R*. R Documentation, 2024.

[4] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. AFIPS Spring Joint Computer Conference, 1967.

[5] C. Clot Denis, *Support de cours – Data Mining Avancé, TD2*. ISFA, 2025.